

既存コンポーネントとヘッダーの修正

ダッシュボード画面と打刻画面それぞれのコンポーネントを修正しています。

また、ルーティング設定を施した直後では表示／非表示が安定しないヘッダーの記述についても、このタイミングで修正しています。

既存コンポーネントの修正

DashboardView.vue の記述を次のように編集します。

```
<template>
  <div>
    <header class="dashboard-header">
      <h1>ダッシュボード</h1>
      <span class="user-welcome">ようこそ、{{ userName }} さん</span>
    </header>

    <section class="announcement-section">
      <h2>お知らせ・通知</h2>
      <div v-if="announcements.length === 0" class="no-announcement">
        現在新しいお知らせはありません。
      </div>
      <div v-else class="announcement-list">
        <div v-for="item in announcements" :key="item.announcementId" class="announcement-card">
          <h3>{{ item.announcementTitle }}</h3>
          <p>{{ item.announcementAbout }}</p>
        </div>
      </div>
    </section>
  </div>
</template>
```

```
<section class="main-actions">
  <button @click="navigateTo('/attendance')" class="btn-main btn-attendance">
    打刻画面へ移動
  </button>
  <button @click="handleLogout" class="btn-main btn-logout">
    ログアウト
  </button>
</section>

<section class="menu-section">
  <h2>各種メニュー</h2>
  <div class="menu-grid">
    <button @click="navigateTo('/schedule-demand')" class="btn-menu">
      予定申請
    </button>
    <button @click="navigateTo('/timechange-demand')" class="btn-menu">
      打刻内容編集申請
    </button>
    <button @click="navigateTo('/profile-edit')" class="btn-menu">
      アカウント情報編集
    </button>

    <button v-if="isAuth === 1" @click="navigateTo('/admin')" class="btn-menu btn-admin">
      【管理者】各種管理画面
    </button>
  </div>
</section>

</div>
</template>
```

```

<script setup>
  import { ref, onMounted } from 'vue';
  import { useRouter } from 'vue-router';
  import apiClient from '../api.js';

  const router = useRouter();
  const announcements = ref([]);

  // 1. 画面表示用のリアクティブ変数を定義（初期値は空文字、または読み込み中を表現）
  const accountId = ref('');
  const userName = ref('ゲスト'); // ローカルストレージにない場合のフォールバック
  const isAuth = ref(0);          // 0:一般, 1:管理者（権限表示がある場合）

  // 2. 画面がマウントされた瞬間にローカルストレージからユーザー情報を復元
  onMounted(() => {
    const userData = localStorage.getItem('user');

    if (userData) {
      try {
        const user = JSON.parse(userData);

        // バックエンドから返ってきたオブジェクトのキー名に合わせて代入します
        accountId.value = user.accountId;
        userName.value = user.userName;    // ★これで「サンプルユーザー」から「本物の名前」に上書きされます
        isAuth.value = user.isAuth || 0;   // もし権限情報も含まれていれば復元

      } catch (e) {
        System.out.error("ユーザー情報のパースに失敗しました", e);
      }
    } else {
      // 3. 【安全対策】もしログイン情報が空っぽでダッシュボードを開かれたら、強制的にログイン画面へ戻す
      console.warn("ログイン情報が存在しません。ログイン画面へリダイレクトします。");
      router.push('/login');
    }

    fetchAnnouncements();
  });

```

```
// お知らせデータの取得
const fetchAnnouncements = async () => {
  try {
    const response = await apiClient.get('/dashboard/announcements');
    announcements.value = response.data;
  } catch (error) {
    console.error('お知らせの取得に失敗しました', error);
  }
};

// 画面遷移ハンドラー
const navigateTo = (path) => {
  router.push(path);
};

// ログアウト処理
const handleLogout = () => {
  if (confirm('ログアウトしますか?')) {
    // セッションやトークンのクリア処理をここに記述
    localStorage.removeItem('token'); // 例
    router.push('/login');
  }
};
</script>
```

```
<style scoped>
.dashboard-container {
  max-width: 800px;
  margin: 0 auto;
  padding: 200px 20px 20px;
  /* ヘッダー等との兼ね合い */
  font-family: sans-serif;
}

.dashboard-header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  border-bottom: 2px solid #34495e;
  padding-bottom: 10px;
  margin-bottom: 20px;
}

/* 上段: お知らせスタイル */
.announcement-section {
  background-color: #f8f9fa;
  padding: 15px;
  border-radius: 8px;
  border-left: 5px solid #3498db;
  margin-bottom: 25px;
}

.announcement-card {
  background: white;
  padding: 10px 15px;
  margin-top: 10px;
  border-radius: 4px;
  box-shadow: 0 2px 4px rgba(0, 0, 0, 0.05);
}
```

```
/* 中段: メインアクション */
.main-actions {
  display: flex;
  gap: 15px;
  margin-bottom: 30px;
}

.btn-main {
  flex: 1;
  padding: 15px;
  font-size: 1.2rem;
  font-weight: bold;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s;
}

.btn-attendance {
  background-color: #2ecc71;
  color: white;
}

.btn-attendance:hover {
  background-color: #27ae60;
}

.btn-logout {
  background-color: #95a5a6;
  color: white;
}

.btn-logout:hover {
  background-color: #7f8c8d;
}
```

```
/* 下段: メニューグリッド */
.menu-grid {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(220px, 1fr));
  gap: 15px;
  margin-top: 15px;
}

.btn-menu {
  padding: 20px;
  font-size: 1rem;
  background-color: #34495e;
  color: white;
  border: none;
  border-radius: 6px;
  cursor: pointer;
  transition: background 0.2s, transform 0.1s;
}

.btn-menu:hover {
  background-color: #2c3e50;
  transform: translateY(-2px);
}

.btn-admin {
  background-color: #e67e22;
}

.btn-admin:hover {
  background-color: #d35400;
}
</style>
```


ヘッダーの表示／非表示の調整

ヘッダーの表示が安定しない最大の原因

結論から申し上げますと、原因は `App.vue` が「今ログインしているかどうか」を判定するタイミングが、画面遷移のスピードと噛み合っていないことにあります。

これまでの `App.vue` は、単純なフラグの変化だけを見てヘッダーの表示（`v-if="isLoggedIn"` など）を切り替えていました。

しかし、`Vue Router` で画面が切り替わる（`/login` から `/dashboard` へジャンプする）とき、`App.vue` 自体は再読み込み（リロード）されず、中央のスクリーンだけが切り替わります。

そのため、以下のすれ違いが起きてしまいます。

- **ログイン直後**：`LoginForm.vue` がストレージに「ログインしたよ」と記録して `/dashboard` に進んでも、親である `App.vue` はストレージの変化に気づかず非表示のままになってしまう。
- **ログアウト直後**：ストレージのデータを消して `/login` に戻っても、`App.vue` がそれに気づかないため、ヘッダーを表示したままにしてしまう。

「定石」的解消策

Vue Router環境でヘッダーの表示・非表示を最も確実に制御する定石は、「現在のURL（ルートパス）を監視する」という方法です。

「ログインしているか」という不確実な状態を見に行くのではなく、「今表示しているURLが `/login` または `/register` のときはヘッダーを隠し、それ以外の画面（ダッシュボードや打刻）のときは無条件でヘッダーを表示する」というルールに変えてしまいます。これなら状態管理のバグが起きません。

今回は `App.vue` を以下のような構造に修正するのが最も妥当です。

`App.vue` の修正

```
<template>
  <div id="app">
    <AppHeader v-if="showHeader" />

    <main class="main-content">
      <router-view />
    </main>
  </div>
</template>
```

```
</div>
</template>
<script setup>
import { computed } from 'vue';
import { useRoute } from 'vue-router';
import AppHeader from './components/AppHeader.vue'; // パスは環境に合わせてください

const route = useRoute();

// ★現在のURL（パス）を監視し、ログイン・登録画面ではない場合のみ true を返す
const showHeader = computed(() => {
  // routeに?（オプションルチューニングという）をつけることで、routeがまだ準備できていない時はエラーにせず、安全に false を返す
  if (!route?.path) return false;

  // URLが '/login' または '/register' の時はヘッダーを「非表示(false)」にする
  return !(route.path === '/login' || route.path === '/register');
});
</script>

<!-- styleは省略しています -->
```

この修正で何が変わるか？

`computed`（算出プロパティ）を使って `route.path` を監視させることで、ユーザーがボタンを押してURLが変わるたび、Vueが自動的に「ヘッダーを出すべきか、隠すべきか」を計算し直してくれます。

これにより：

1. ログインが成功してURLが `/dashboard` になった瞬間に、ヘッダーがパッと出現します。
2. ログアウトしてURLが `/login` に戻った瞬間に、ヘッダーがスッと消え去ります。

まずはこの `App.vue` の「URL監視方式」への切り替えを確認してみてください。ヘッダーの出し入れに関する1番目と2番目の問題は、これだけで解決します。

打刻画面：ラベル設定の最新化

「打刻画面のラベル設定（`labelSettings`）が反映されない問題」の解決に取り掛かりましょう。

ここまでの実装状況では、`AttendanceBoard.vue` の中でモード（`attendance` や `room` など）に応じたラベル切り替えロジックが用意されているものの、それが画面に反映されていない状態になっています。

この問題が起きる原因は、主に以下の2つのいずれか（または両方）にあります。

1. **現在のモード（`currentMode`）を指定する変数が存在しない、または初期値がセットされていない**
`labelSettings` 自体はただの辞書（定義）であるため、画面側が「今、どのモード（例: `attendance`）で表示すべきか」を追跡するためのリアクティブ変数（例: `currentMode`）が欠落しているか、空っぽのままになっている可能性があります。
2. **テンプレート（HTML）側で `labelSettings` の参照方法が間違っている**
HTML側で直接 `{{ labelSettings.attendance.inActive }}` のように固定で書いてしまっているか、あるいは動的に切り替えるための `labelSettings[currentMode.value]?.inActive` といった記述と噛み合っていないケースです。

AttendanceBoard.vue の修正手順

画面が現在のモードを正しく認識し、それに応じたラベル（「勤務中」「入室中」など）を動的に出し分けられるよう、スクリプトとテンプレートを修正します。

1. スクリプト部分（<script setup>）の修正

「現在のモード」を保持する変数（currentMode）を定義し、デフォルトとして attendance（または仕様に合わせた初期モード）をセットします。

```
<script setup>
import { ref, onMounted, computed } from 'vue';
import apiClient from '../api';
const history = ref([]);

// ローカルストレージから取得したIDを保持するリアクティブ変数
const loggedInAccountId = ref('');
const userName = ref('');
```

```
// 現在の表示モードを管理する変数（初期値を 'attendance' に設定）
const currentMode = ref('attendance');

// 履歴データを格納する配列
const attendanceHistory = ref([]);
/**
 * バックエンドから打刻履歴を取得する関数
 * @param {String} id - ログインユーザーのaccountId
 */
const fetchAttendanceHistory = async (id) => {
  if (!id) return;
  try {
    // バックエンドの履歴取得API (GET /api/attendance/history/{accountId}) を叩く
    const response = await apiClient.get(`/attendance/history/${id}`);

    // 取得したデータを履歴配列にセットする
    attendanceHistory.value = response.data;
  } catch (error) {
    console.error('打刻履歴の取得に失敗しました:', error);
  }
};
```

```
onMounted(() => {
  // LoginForm.vue が保存した 'user' データを読み出す
  const userData = localStorage.getItem('user');
  if (userData) {
    const user = JSON.parse(userData);
    loggedInAccountId.value = user.accountId; // ★これで正しいIDがセットされます
    userName.value = user.userName;
  } else {
    console.error("ユーザー情報が見つかりません。");
  }

  // 既存の初期化処理（履歴の取得など）があれば、loggedInAccountId.value を使って呼び出す
  if (loggedInAccountId.value) fetchAttendanceHistory(loggedInAccountId.value);
});

// モードに応じたラベルの定義
const labelSettings = {
  attendance: { inActive: '勤務中', outActive: '未出勤', inAction: '出勤', outAction: '退勤' },
  room:       { inActive: '入室中', outActive: '退室済', inAction: '入室', outAction: '退室' },
  session:    { inActive: '出席中', outActive: '退席中', inAction: '出席', outAction: '退席' }
};
```



```
// 現在選択されているモードのラベル群を返す
const currentLabels = computed(() => {
  return labelSettings[currentMode.value] || labelSettings.attendance;
});

// 現在のモード (propsから受け取る。デフォルトは 'attendance')
const labels = computed(() => labelSettings[props.mode || 'attendance']);

const status = ref('CLOCKED_OUT');

// 現在の就業ステータス
const currentStatus = ref(0)

// ステータス取得API
const fetchStatus = async () => {
  if (!loggedInAccountId.value) return;
  try {
    const response = await apiClient.get(`/attendance/status?accountId=${loggedInAccountId.value}`);
    status.value = response.data; // バックエンドから "CLOCKED_IN" または "CLOCKED_OUT" が届く
  } catch (error) {
    console.error('ステータス取得失敗', error);
  }
};

// 履歴取得
const fetchHistory = async () => {
  try {
    const response = await apiClient.get(`/attendance/history/${loggedInAccountId.value}`);
    console.log("バックエンドから届いた生の履歴データ:", response.data);

    history.value = response.data;
  } catch (error) {
    console.error('履歴取得失敗', error);
  }
};
```

```
// 履歴をペアリングして直近10件分を返すロジック
const pairedHistory = computed(() => {
  // historyが存在しない、または空の場合は即座に空配列を返す
  // if (!props.history || !Array.isArray(props.history)) return [];

  // props.history ではなく、history.valueを見に行くようにします
  if (!history.value || !Array.isArray(history.value)) return [];

  const result = [];
  const sortedHistory = [...history.value].reverse(); // 存在する時だけスプレッド演算子でコピーを作る

  for (let i = 0; i < sortedHistory.length; i++) {
    const record = sortedHistory[i];

    if (record.type === 'clock-in') {
      // 次のレコードが clock-out かつ同じペアになるべきものかチェック
      const nextRecord = sortedHistory[i + 1];
      if (nextRecord && nextRecord.type === 'clock-out') {
        result.push({
          date: formatDate(record.createdAt),
          inTime: formatTime(record.createdAt),
          outTime: formatTime(nextRecord.createdAt)
        });
        i++; // 退室レコード分をスキップ
      } else {
        // 退室がない場合は進行中として表示
        result.push({
          date: formatDate(record.createdAt),
          inTime: formatTime(record.createdAt),
          outTime: '-'
        });
      }
    }
  }

  // 最新の10件を返す（再び最新が上に来るように逆転させる）
  return result.reverse().slice(0, 10);
});
```

```
// emit の定義
const emit = defineEmits(['refresh-history']);

// 打刻処理
const punch = async (type) => {
  try {
    // 1. クエリパラメータ形式から、JSON ボディ形式に変更
    // バックエンドの @RequestBody Map<String, String> request と一致させます
    console.log("accountId: " + loggedInAccountId.value + ", type: " + type);
    await apiClient.post('/attendance/punch', {
      accountId: loggedInAccountId.value, // route設定後はpropsではなく、ローカルストレージから復元した変数を指定
      type: type // 'CLOCK_IN' または 'CLOCK_OUT'
    });

    // 2. 打刻が成功したら、ステータスと履歴を更新
    // fetchStatus は「現在の状態：入室中」などの表示更新
    // fetchHistory は「最近の履歴」リストの更新を担当します
    await fetchStatus();
    await fetchHistory();

    // 3. 親（App.vue）側でも履歴を管理している場合は、イベントを飛ばす
    // ※もしApp.vueのattendanceHistoryを更新したい場合に有効です
    emit('refresh-history');

    // 打刻が成功したので、最新の履歴を再取得して画面をパッと更新する！
    fetchAttendanceHistory(loggedInAccountId.value);

    // ステータス（現在入室中か退室中か）の再取得関数があればそれも呼ぶ
    if (typeof fetchStatus === 'function') fetchStatus();

    // 成功のフィードバック（任意）
    console.log(`${type} 成功`);
  } catch (error) {
    console.error('打刻エラー:', error);
    alert('打刻に失敗しました。');
  }
};
```

```
// 日時フォーマット用の補助関数
const formatDate = (dateStr) => new Date(dateStr).toLocaleDateString('ja-JP');
const formatTime = (dateStr) => new Date(dateStr).toLocaleTimeString('ja-JP', { hour: '2-digit', minute: '2-digit', second: '2-digit' });

onMounted(() => {
  fetchStatus();
  fetchHistory(); // 画面が開いたときに履歴も読み込む
});
</script>
```

2. テンプレート部分（<template>）とスタイル（<style scoped>）の修正

HTML側（ボタンやステータス表示部分）で `currentLabels` を使って文字を動的に表示させます。ヘッダーの修正でも大活躍した `v-if / v-else` と組み合わせることで、洗練された見た目になります。

```
<template>
  <div class="attendance-board">
    <div class="attendance-board">
      <h3>ようこそ、{{ userName }} さん</h3>

      <div class="status-display">
```

```
<p>現在の状態 :
  <span :class="status === 'CLOCKED_IN' ? 'status-in' : 'status-out'">
    {{ status === 'CLOCKED_IN' ? currentLabels.inActive : currentLabels.outActive }}
  </span>
</p>
</div>

<div class="punch-actions">
  <button
    v-if="status === 'CLOCKED_OUT'"
    @click="punch('CLOCK_IN')"
    class="btn btn-in"
  >
    {{ currentLabels.inAction }}する
  </button>

  <button
    v-else
    @click="punch('CLOCK_OUT')"
    class="btn btn-out"
  >
    {{ currentLabels.outAction }}する
  </button>
</div>
```

```
<div class="mode-switcher" v-if="status === 'CLOCKED_OUT'">
  <button @click="currentMode = 'attendance'" :class="{ active: currentMode === 'attendance' }">勤怠モード</button>
  <button @click="currentMode = 'room'" :class="{ active: currentMode === 'room' }">入退室モード</button>
  <button @click="currentMode = 'session'" :class="{ active: currentMode === 'session' }">出席退席モード</button>
</div>

</div>

<hr class="divider">

<div class="history-section">
  <h3>最近の履歴（直近10件）</h3>
  <ul class="history-list">
    <li v-for="(entry, index) in pairedHistory" :key="index" class="history-item">
      <span class="date">{{ entry.date }}</span>
      <span class="time">Entry: {{ entry.inTime }}</span>
      <span class="separator"> ~ </span>
      <span class="time">Exit: {{ entry.outTime }}</span>
    </li>
  </ul>
</div>

</div>
</template>
```

次の記述を `<style scoped>` に追加してください

```
/* モード切り替えボタンのコンテナ（親要素） */
.mode-switcher {
  display: flex;           /* ボタンを横並びにする */
  justify-content: center; /* 中央寄せにする */
  gap: 12px;              /* ★ボタンとボタンの間の余白（隙間）を適度にとる */
  margin-top: 25px;        /* 打刻エリアとの上下のディスタンス */
  margin-bottom: 20px;
  flex-wrap: wrap;         /* 画面幅が狭いときは自動で綺麗に折り返すようにする安全策 */
}

/* モード切り替えボタン単体のデザイン */
.mode-switcher button {
  background-color: #f8f9fa; /* 普段は主張しすぎない上品な薄いグレー */
  color: #495057;
  border: 1px solid #ced4da;
  padding: 10px 16px;       /* ★上下10px、左右16pxの内側余白をとり、押しやすい大きさに */
  border-radius: 6px;       /* 角を少し丸めてモダンな印象に */
  font-size: 0.95rem;
  font-weight: 500;
  cursor: pointer;
  transition: all 0.2s ease; /* マウスを乗せた時やクリックした時の変化を滑らかにする */
}

/* マウスホバー時のエフェクト */
.mode-switcher button:hover {
  background-color: #e9ecef;
  border-color: #adb5bd;
}

/* ★オマケ：現在アクティブ（選択中）なモードのボタンを強調するスタイル */
/* もしHTML側で「:class="{ active: currentMode === 'attendance' }"のように制御する場合に輝きます */
.mode-switcher button.active {
  background-color: #007bff;
  color: white;
  border-color: #007bff;
  box-shadow: 0 2px 4px rgba(0, 123, 255, 0.2);
}
```

修正のポイント

- `computed` (`currentLabels`) による自動切り替え

`currentLabels.inActive` のように指定しておくだけで、仮に「勤務モード」から「入退室モード」に `currentMode` がパチッと切り替わった際、画面上の文字が連動して一瞬で「勤務中」から「入室中」へと自動翻訳されるようになります。

- ボタン名も連動

ボタンのテキストも `{{ currentLabels.inAction }}` としているので、「出勤する」や「入室する」へと綺麗に可変対応します。

現在の `AttendanceBoard.vue` の該当箇所（ラベルの定義と、それを表示しているHTML部分）を、この `currentLabels` を使う形に書き換えてみてください。